基于 Python 语言实现自动化压力测试系统的设计研究

赵芮，徐小冬，蒋莉，郑斌，阎玲
（江苏方天电力技术有限公司，江苏南京，211010）

摘要：随着科技发展现代的网络通信设备在组成上变得越来越复杂，且工作量较大，要在较短的时间内完成测试工作，给测试系统带来了巨大的压力。Python 一种开源通用的计算机编程语言，能够使语法开发时间大幅缩短，而且是语法易读简洁，能够更好地开展维护工作，扩展模块也非常的丰富，使其便捷性和扩展性大幅提高。

关键词：自动化；压力测试系统；Python；SIPp
DOI:10.16520/j.cnki.1000-8519.2020.13.036

Design and Research of Automatic Stress Testing System Based on Python Language

Zhao Rui，Xu Xiaodong，Jiang Li，Zheng Bin，Yan Ling
(Jiangsu Fangtian Power Technology Co.,Ltd .,Nanjing Jiangsu,211010)

Absrtact: With the development of science and technology, the modern network communication equipment becomes more and more complex in composition, and the workload is large, so it is necessary to complete the test work in a short time, which brings great pressure to the test system. Python an open source common computer programming language, it can greatly shorten the grammar development time, and it is easy to read and concise, can better carry out maintenance work, the extension module is also very rich, so that its convenience and scalability greatly improved.

Keywords: automation; stress testing system; Python;SIPp

0 引言

在 Python 以及 SIPp 自动化压力测试系统，可以自动化地完成人工测试和编排，使工作效率大幅增强，而且还有效控制了成本投入。在 Python 以及 SIPp 基础上对自动化压力测试系统进行设计，使传统测试系统存在的问题得到有效解决，更保证了测试效率，控制了成本投入。

1 系统组成

1.1 系统的硬件结构

测试管理子系统，被测设备以及 SIPp 测试节点是自动化压力测试系统的主要组成，其连接示意图参见图 1 所示。充分分析测试模块和测试设备具有的压力，能够对个 SIPp 测试节点进行有效的拓展，可以集中化的管理测试系统，对任务开展分配工作，基于交换机以及路由器和网络前提下，连接所有的设备，该系统接口使用网络接口，不但可以对 SIPp 节点进行管理，对模拟负荷进行启动和停止，在实际测试时，对负荷大小合理的调整，对测试设备能够有效的进行监控，对其高压条件下运行指标和状态进行判断。

1.2 系统的软件结构

系统组成主要包括，管理模块配置和 SSH 以及 SIPp 和采集设备参数模块与生成报告模块。测试系统当中的各种预测参数通过管理配置模块来完成。Linux 系统是 SIPp 工具主要运行系统，在登陆系统过程当中，应当借助 SSH 协议来对 Linux 服务器进行登录，合理的启动 SIPp 或者做出停止等，SSH 接口必须有相应的工具支持。在 SSH 模块中封装相映的连接登录命令获取和执行命令等操作，便于模块中应用其他工具。

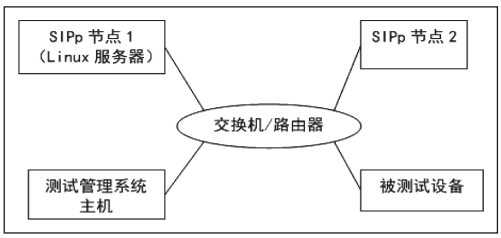

图 1　系统硬件连接示意图

2 运行环境

2.1 Python 扩展模块 paramiko 的安装

Ssh 与 sftp 在 paramiko 模块，执行命令以及上传下载文件功能，通过远程登录服务器来实现，这一功能是第三方的 Python 扩展包，在应用过程当中应当进行相应的安装。在安装 paramiko 主要的程序：对 window 版本 Python2.7 进行安装下载，并对 PyCrypto2.6 for Python 2.7 64bit 下载安装，同时对 ecdsa-0.10.tar.gz 安装下载，再次安装下载 paramiko.tar.gz，成功之后，进入到 python，将 import paramiko 输入其中，信息正确之后提示成功安装。

2.2 安装 SIPp 工具，编写测试脚本

该工具是对 SIP 协议性能进行测试的开源软件，在 Linux 平台下进行运行，对测试场景脚本首先进行编排，对收

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

发 rc SIP 信息科学控制，能够对用户终端进行模拟，还能够模拟通信服务器，对 SIP 协议流程交互进行设备测试于完成，在各种脚本配置下，能够对 SIP 协议那内的 UAC 和 UAS 充当，同时对 SIP 会话进行串行地发起以及并发，并动态的输出测试统计数据，因其具有开源特点和强大的性能，在通讯设备当中 SIPp 用十分的广泛，可以有效地测试通信设备压力，实际的步骤为，通过 UAC 进行主叫发起，对设备进行测试，被叫 UAS。SIPp 担任 UAC 和 UAS 任务，安装 SIPp 过程当中非常的简单，将代码从开源网站进行下载，并在 Linux 服务器开展拷贝工作，编译工作完成之后即可，测试场景脚本过程当中，合理设计于编写用户号码脚本，调试收发信息流程。

3 设计与实现系统模块

3.1 设计与实现 SSH 接口模块

利用二次封装 paramiko 接口的第三方模块，在简单的接口上封装繁琐的协议函数，对命令接口进行执行，对文件接口以及文件下载接口进行执行。SSH 接口模块分为以下几个部分。

3.1.1 SSH 初始化接口

首先导入 paramiko 模块

```
import paramiko
```

然后定义 SSH 封装类及其初始化的相应函数：

```
class ftpclient(object):
def __init__(self):
def sftp_connect(self, hostname, port, username, passwd):
def ssh_connect(self, hostname, port, username, passwd):
def cmd_ssh(self, cmd):
def upload_file(self, localfile,localfile):
def download_file(self,remotefile,localfile)：
return;
```

3.1.2 SSH 命令执行接口

```
ftp = ftpclient()
```

3.1.3 SFTP 文件传递操作接口

```
sftp=ftpclient()// 上传文件 // 下载文件
```

3.2 SIPp 控制模块

3.2.1 对 SIPp 脚本进行启动

在 Linux 服务器上对于 SIPp 工具运行进行预设，并将此程序安装好之后，对收发 sip 消息流程脚本文件进行合理编排，同时编排好用户号码本脚本文件，并启动 linux shell 命令。上述的这些代码，将 SIPp_cmd 字符串连代码进行定义，启动 SIPp，这一命令主要对定义的 reg.xml 收发消息序列进行执行，并通过 Reg_user.csv，迭代定义当中的用户号码，基于特定速率下，通过后台方式开展执行工作。对 ssh 接口模块执行命令函数进行调用，对此命令进行执行，通过 SIPp 模块控制，对此节点上一个测试流程进行启动。

3.2.2 实际测试时，对 sip 发送信息速率动态的进行调整

测试压力时，对压力程度开展周期性的变化，主要是指需要变化发送消息速率的 SIPp，借助 -r 参数对消息初始发送速率进行指定，并按照这一速率持续的运行后续 SIPp。具体通过人工开展测试过程当中，基于运行的 SIPp 界面，将“+”、“-”、“*”、“/”键入其中，对消息发送速率合理的进行调整，通过自动化测试，减少人为因素带来的影响，同时在后台模式上运行 SIPp。所以需要通过别的途径再对 SIPp 速率进行相应的调整，借助 SIPp 有关文档，发现运行过程当中的 SIPp 实例，有 UDP 协议管理端口预留的，通过 8888 开始，每进行一个实例增加，也相应的进行一个端口顺次增加，SIPp 实例第一个运行的管理端口为 8888，8889 试运行第二个管理端口，并以此进行类推，将 UDP 消息向该端口进行发送，实现 SIPp 实例运行控制，应当对发送第一个 SIPp 实例消息速率进行调整，使其达到 1000 次每秒，主要的。Lansing shell 命令响应为，echo“cset rate 2000”>/dev/udp/127.0.0.1/8888，在 ssh 接口模块，将 python 代码进行 SIPp 节点发送。

3.2.3 停止 SIPp 脚本

测试压力完成之后，应当对发送消息的 SIPp 节点进行自动停止，在此过程当中存在的方式主要为两种，一种为优雅的方式，一种为强制的方式，第一种方式是通过发送 SIPp 结束命令来实现，测试流程通过 SIPp 缓慢地结束，不再对新的消息进行发送，对已经发出的消息全面等待，响应之后，对流程进行全面关闭，而后一种方式主要是指不考虑信息发送有无得到响应，通过强制手段来停止 SIPp 实例。

3.3 数据采集及分析模块

在实际测试时和完成测试之后，测试数据需要通过设备进行采集，对设备运行状态进行判断，了解企业是否存在故障问题。系统当中主要利用的模型为 TCP Server/Client，对测试设备网管接口进行登录，并对采集数据命令开展执行工作，将相应的运行实际数据进行获得，并通过一个类接口封装，主要的接口包括登录初始化以及网管执行命令接口等。

4 结束语

在软件开发过程当中，测试软件是非常重要的一项工作内容，对确保软件质量有着非常重要的意义，需求设计以及编码等最终的检查工作，再 Python 以及 SIPp 制动压力测试系统下，通过自动化测试步骤编排，将传统测试工作当中存在的不足有效克服，工作效率大幅增强，而且成本还得到了很好的控制。

参考文献

[1]Mark Lutz. 邹晓，译 .Python 编程 [M]. 北京：中国电力出版社，2015.

[2] 黄建军，李宥谋，刘婧，等 . 基于 Python 语言的自动化测试系统的设计与实现 [J]. 现代电子技术，2017(2)：39-42.

[3] 何晨，曹毅 . 基于 SIPp 的 SIP 协议性能测试研究与应用 [J]. 电子技术与软件工程，2014(15)：43-45.

(C)1994-2024 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net